

Fine-Tuning Qwen3-27B for C-to-Rust Code Translation: A Three-Stage Curriculum of Pretraining, Debugging-Aware SFT, and Task-Specific SFT

Pu Zhao¹, Changdi Yang¹, Yixiao Chen¹, Yi Gao¹, Yifan Cao²,
Haochen Zeng³, and Yanzhi Wang¹

¹Northeastern University

²EmbodimentX Inc

³Aibao LLC

Abstract

Translating C code into safe, idiomatic Rust is a longstanding software-engineering goal because it can eliminate entire classes of memory-safety vulnerabilities while preserving the functional behavior of legacy systems. Large language models (LLMs) have shown promise for this task but typically underperform when applied off-the-shelf, since general-purpose pretraining rarely emphasizes idiomatic Rust generation, cross-language semantic equivalence, or the ability to reason about and repair compiler/runtime feedback. In this report we describe a three-stage fine-tuning curriculum applied to Qwen3-27B that is designed to progressively specialize the model for the C-to-Rust (C2Rust) translation task: (1) continued pretraining on Rust-centric corpora to strengthen the model’s prior over idiomatic Rust syntax and standard-library usage; (2) supervised fine-tuning (SFT) on the `microsoft/Verus.Training.Data` dataset to instill debugging and self-repair behavior over Rust code; and (3) task-specific SFT on paired C/Rust solutions derived from LeetCode problems to teach direct semantic translation. We evaluate the resulting model using the agentic, static-analysis-guided verification framework of SACTOR, which performs structure-aware, two-phase (unidiomatic $\rightarrow$ idiomatic) translation with foreign-function-interface (FFI)-based end-to-end (E2E) testing. We report success rate, idiomaticity (Clippy lint counts, unsafe-code fraction), and failure-mode analyses, and compare our fine-tuned model against baseline Qwen3-27B and other LLMs evaluated under the same framework.

Model: <https://huggingface.co/moxin-org/C2Rust>

Code: <https://github.com/moxin-org/C2Rust>

Contents

1	Introduction	2
1.1	Contributions	4
2	Related Work	4
3	Base Model	6
3.1	Architecture	7
3.2	Tokenizer and Licensing	8
3.3	Fine-Tuning Hardware	8
3.3.1	Distributed Training Strategy: DeepSpeed ZeRO Stage 3	8

4	Training Data and Curriculum	9
4.1	Stage 1: Rust-Focused Continued Pretraining	9
4.1.1	Data Composition	9
4.1.2	Concatenation Design	11
4.1.3	Training Configuration	12
4.2	Stage 2: Debugging-Aware SFT on Verus_Training_Data	12
4.2.1	Dataset Origin and Task Nature	12
4.2.2	Data Synthesis Pipeline (VeruSyn)	12
4.2.3	Quality Control	13
4.2.4	Relevance to This Work	13
4.2.5	Dataset Composition	14
4.2.6	Training Configuration	14
4.3	Stage 3: C2Rust Task-Specific SFT on the C2Rust-Moxin Dataset	14
4.3.1	Data Quality Considerations	15
4.3.2	Training Configuration	15
5	Evaluation Framework	15
5.1	SACTOR Pipeline	16
5.2	Evaluation Benchmark	16
5.3	Verification and Repair	17
5.4	Inference Configuration	17
5.5	Failure Analysis	17
5.6	Evaluation Metric	18
6	Results	18
6.1	Overall SR and Baseline Comparison	18
6.2	General Coding Capability: SWE-bench Verified	19
7	Conclusion	20

1 Introduction

C remains the implementation language of choice for a large fraction of performance-critical and systems-level software, but its manual memory management is a persistent source of vulnerabilities such as buffer overflows, use-after-free errors, and memory leaks. Rust addresses these issues through a compile-time ownership and borrowing model that guarantees memory safety without a garbage collector, and it has seen growing adoption in security-sensitive contexts (e.g., the Linux kernel, Firefox). Migrating existing C codebases to Rust therefore has clear practical value, but manual translation is slow, expensive, and error-prone, motivating automated and LLM-assisted translation approaches.

Rule-based tools such as C2Rust [13] generate syntactically valid Rust by transliterating C ASTs, but the resulting code is heavily unsafe and unidiomatic. Recent work has explored LLM-driven translation with various forms of verification and feedback (e.g., fuzzing, symbolic execution, FFI-based E2E testing) to improve both correctness and idiomatity [64]. However, general-purpose LLMs [9, 48, 41, 25, 42, 60, 63, 61, 62] are not natively specialized for this task: they may lack deep familiarity with idiomatic Rust patterns, may not reliably reason about ownership/lifetime semantics, and may struggle to use compiler and test feedback to iteratively repair their own output.

The C2Rust setting we study is best situated within the much broader trajectory of LLMs applied to code more generally. Autoregressive, decoder-only language models trained on very large source-code corpora

have become the dominant approach to automated program synthesis, translation, and repair, and progress in that broader literature has come from three largely complementary directions. The first is simply scaling: successive generations of code-oriented LLMs have grown in both pretraining-corpus size and parameter count, and general software-engineering benchmarks such as SWE-bench Verified [16] (which we also use in §6.2) have tracked steady gains from this scaling alone. The second is post-training: instruction tuning and, increasingly, reinforcement learning with verifiable rewards (i.e., using compiler or test-execution outcomes directly as a training signal) have proven effective at aligning a fixed-capacity model’s outputs with the kind of structured, executable feedback that code tasks naturally provide. The third is inference-time scaffolding: agentic tool use, retrieval of relevant context, and iterative repair loops let a model make repeated, feedback-conditioned generation attempts rather than committing to a single one-shot output. Our own three-stage curriculum and our adoption of SACTOR’s agentic, repair-driven evaluation loop are best understood as instances of, respectively, the second and third of these directions, specialized to the narrower C-to-Rust translation task rather than as departures from the broader LLM-for-code literature.

A more recent and, so far, largely separate line of work explores diffusion-based language models (dLLMs) as an alternative generative paradigm to autoregressive decoding for code. Rather than producing tokens strictly left-to-right, masked or discrete diffusion models iteratively denoise an entire sequence, refining many token positions in parallel across a fixed number of denoising steps; this global, non-causal generation process has been motivated for code specifically on the grounds that writing code often involves non-sequential, back-and-forth revision rather than strict left-to-right composition. Open-weight dLLMs such as LLaDA [27] and Dream [55], together with code-specialized variants such as DiffuCoder [7] and Dream-Coder [52], have shown that diffusion-based decoding can approach the quality of similarly sized autoregressive models on coding benchmarks, while commercial-scale systems such as Mercury [14] and Gemini Diffusion [8] have demonstrated diffusion-based code generation that is competitive with leading autoregressive code models while offering substantially faster generation. At the same time, the dLLM ecosystem remains considerably less mature than that of autoregressive LLMs: serving optimizations that are now standard for autoregressive decoding, such as key-value caching, speculative decoding, and prefix caching, have no direct analogue in the diffusion setting, and dLLMs typically must commit to an output length in advance rather than terminating generation naturally via an end-of-sequence token. We view diffusion-based code models as a plausible complementary direction for future C2Rust work rather than as directly applicable to the present study: notably, the idiomatic-refinement phase of the SACTOR pipeline we adopt is itself a form of iterative, whole-program revision rather than strictly incremental generation, a structural resemblance to denoising that could make dLLMs a natural fit for that specific sub-task, though we leave an empirical investigation of this possibility to future work. Throughout this report, our base model and all three curriculum stages retain the standard autoregressive, next-token-prediction formulation, and we do not otherwise revisit diffusion-based decoding below.

This report documents an effort to specialize **Qwen3-27B** for the C2Rust translation task via a three-stage fine-tuning curriculum:

- Stage 1: Rust-focused continued pretraining.** We continue pretraining the base model on a corpus of Rust code and text collected from multiple existing Rust-capable models/sources, with the goal of strengthening the model’s general command of Rust syntax, idioms, and standard-library usage prior to any task-specific tuning.
- Stage 2: Debugging-oriented SFT.** We fine-tune on `microsoft/Verus_Training_Data`, an instruction dataset centered on debugging Rust code (e.g., diagnosing and repairing compiler errors, verification failures, or semantic bugs), with the aim of teaching the model to interpret error signals and produce targeted repairs – a skill directly relevant to the iterative, feedback-driven verification loop used at evaluation time.

Stage 3: C2Rust task-specific SFT. We fine-tune on instruction data constructed from LeetCode problems for which both a C solution and a corresponding Rust solution are available, directly teaching the source $\rightarrow$ target translation mapping the model must perform at inference time.

We evaluate the final model using the agentic evaluation framework described by SACTOR [64], which decomposes translation into an interface-preserving *unidiomatic* phase and a refinement-based *idiomatic* phase, each verified via FFI-linked E2E tests against the original C program, with structured compiler/test feedback driving up to N repair attempts. Our model with 27B parameters demonstrates a success rate of 87.2%, outperforming MiniMax-M2.5 and GLM-5 with hundreds of billions of parameters.

1.1 Contributions

- A three-stage curriculum – Rust pretraining $\rightarrow$ debugging SFT $\rightarrow$ C2Rust SFT – for specializing a general-purpose 27B-parameter LLM to the C2Rust translation task.
- An evaluation of the resulting model under the SACTOR agentic verification framework, reporting success rate and idiomaticity metrics.

2 Related Work

Rule-based and static C-to-Rust translation. C2Rust [13] performs AST-level transliteration from C to Rust, producing code that compiles but relies heavily on unsafe blocks and raw pointers, yielding low readability and limited safety benefit; Hong and Ryu’s survey [10] traces the broader landscape of automatic C-to-Rust translation techniques that have followed. Subsequent static/rule-driven refinements target specific unidiomatic patterns left behind by transliteration – for example, replacing C-style output parameters with algebraic-data-type return values, translating C’s tagged unions into Rust’s enum-based tagged unions, and automatically deriving lock APIs for concurrent C code – but remain fundamentally rule-driven and struggle to generalize to complex, real-world code. Hong and Ryu also explore a hybrid point in this space, using an LLM specifically to resolve the type-migration step of C-to-Rust translation while keeping the surrounding pipeline rule-based [11].

Single-shot and prompting-based LLM translation. A first wave of LLM-driven approaches treats C-to-Rust translation largely as a prompting problem, optionally combined with post-hoc verification signals such as fuzzing, symbolic execution, or dynamic-analysis-guided repair. SafeTrans [6] evaluates six LLMs across nearly 16,000 translation attempts, combining basic compiler-feedback repair with a few-shot guided repair phase keyed to specific Rust error categories (trait-implementation failures, borrow-checker violations), and additionally studies whether memory-safety vulnerabilities in the original C code persist after translation. C2SaferRust [28] combines LLM-driven translation with neurosymbolic techniques aimed at reducing residual unsafe code, though SACTOR’s own analysis notes it still produces Rust with numerous unsafe blocks [64]. VERT [54] targets verified-equivalent transpilation, using LLMs as few-shot learners to produce Rust translations whose behavioral equivalence to the source can be formally checked, though it is reported to struggle on programs with complex data structures [64].

Verification- and feedback-guided translation. Closest to our own evaluation setting, SACTOR [64] proposes a two-stage, structure-aware pipeline (interface-preserving unidiomatic translation, followed by idiomatic refinement) guided by static analysis and verified end-to-end via FFI linking with the original C program; a SPEC-driven harness generator bridges ABI mismatches introduced by idiomatic refactoring. We adopt this framework, unmodified, as our evaluation harness so that our fine-tuned model’s performance

can be compared against the LLMs benchmarked in that work (e.g., GPT-4o, Claude 3.5 Sonnet, Gemini 2.0 Flash, Llama 3.3 70B, DeepSeek-R1). AdaTrans [24] similarly builds a generate-verify-repair loop around compiler feedback, but adapts its repair strategy to the specific category of failure observed (syntax vs. ownership vs. behavioral errors) via an error-stratified retrieval-augmented generation mechanism, reporting a 95.51% compilation pass rate and 81.09% solve rate on a 104-problem algorithmic benchmark; Syzygy [43] takes a related dual code-test translation approach, jointly translating a C program and its tests to Rust and using dynamic analysis to validate the pair.

Agentic and multi-step translation systems. A further line of work frames C-to-Rust translation as a multi-step or agentic process rather than a single prompt-and-repair loop. Sim et al. [44] propose a search-based, multi-trajectory LLM agent that explores several candidate translation paths per program rather than committing to a single greedy repair sequence. AlphaTrans [12] combines neural translation with symbolic validation at the repository level rather than per-function, aiming for compositional correctness across an entire codebase. Both are complementary to SACTOR’s own two-phase design in that they explore alternative strategies for structuring the search over candidate translations under verifier feedback.

Project- and repository-level translation. Because a single function or program is a limited proxy for real-world migration difficulty, several recent systems target project-scale translation directly. EvoC2Rust [49] first generates a compilable Rust “skeleton” of type-checked function stubs for an entire project, then incrementally fills in each function and repairs remaining compilation errors via static analysis, avoiding a conventional transpiler-preprocessing step. His2Trans [50] extends this skeleton-first idea with a self-evolving historical-retrieval mechanism, mining previously completed migrations to infer idiomatic API correspondences and reduce repeated repair overhead on unseen projects by an order of magnitude. PtrTrans [56] instead constructs a C-Rust pointer knowledge graph encoding ownership, mutability, nullability, and lifetime information extracted via program analysis, injecting these facts into LLM prompts to guide project-level, ownership-compliant generation. These project-level systems are the closest existing analogs to the `projects/` granularity of the C2Rust-Moxin dataset described in §4.3 (though, as noted there, our Stage 3 SFT itself draws only on the `functions/` and `programs/` granularities).

Benchmarks and empirical studies. Beyond CRust-Bench [18] and the TransCoder-IR [46]/CodeNet [31] datasets used in the SACTOR evaluation protocol, several other benchmarks and studies characterize LLM translation quality more broadly. CodeTransOcean [53] provides a large multilingual benchmark spanning many source/target language pairs (not limited to C/Rust), useful for situating C-to-Rust performance within the broader landscape of LLM code translation capability. Ou et al. [30] introduce a benchmark specifically targeting repository-level translation into Rust, complementing CRust-Bench’s function-level focus. Li et al. [22] report a user study of practitioners translating C to Rust, providing qualitative grounding for what translation errors and idioms matter most to human developers – a useful check against benchmarks that measure only automated pass/fail outcomes.

Instruction tuning and debugging-aware training. Instruction fine-tuning on debugging/repair data has been used elsewhere to improve models’ ability to consume compiler diagnostics and test failures and to produce targeted patches; the error-stratified repair strategies used by AdaTrans [24] and the historical-repair-pattern reuse in His2Trans [50] (discussed above) are inference-time analogs of the training-time debugging skill we aim to instill in Stage 2. We adopt microsoft/Verus Training Data [4], a Rust-centric formal-proof-debugging dataset (§4.2), as an intermediate-stage curriculum designed to transfer this self-repair capability to the C2Rust setting, where iterative feedback (from the idiomatic/unidiomatic verifiers) is central to the evaluation protocol.

General-purpose LLMs for code, beyond C-to-Rust. The C2Rust-specific literature surveyed above sits inside a much larger body of work applying LLMs to code more broadly, and several trends from that literature recur in our own design choices. Base-model scaling and code-specific continued pretraining – the same strategy we use in Stage 1 (§4.1) – underlie essentially every strong general-purpose code model, from early code-pretrained encoder-decoder and decoder-only models through to today’s frontier systems; StarCoderData [1], one of the seven Stage 1 sources, is itself the pretraining corpus behind an entire family of open code models rather than a C2Rust-specific artifact. Reinforcement learning with executable feedback – compiler success, test-suite pass rates, or, in the case of Verus, formal-verifier acceptance – has similarly become a standard post-training ingredient for code models generally, not only for translation systems; the VeruSyn pipeline underlying our Stage 2 data (§4.2) is one instance of this broader pattern, in which a model is trained directly against a formal or executable success signal rather than against human preference labels alone. Finally, general software-engineering benchmarks such as SWE-bench [16] were developed specifically to measure this broader, task-agnostic notion of coding competence (issue localization, multi-file editing, real repository context) as a check against benchmarks – including our own C2Rust evaluation – that isolate a single, narrower translation task; we report SWE-bench Verified results for exactly this reason in §6.2.

Diffusion language models for code generation. A separate line of work replaces autoregressive, left-to-right decoding with masked or discrete diffusion, in which a model iteratively denoises an entire token sequence rather than committing to tokens strictly in order [38, 59, 35, 37, 40, 47, 36, 39, 19]. CodeFusion [45] was the first system to combine a diffusion objective with code generation, but was restricted to small-scale models and comparatively simple synthesis tasks. More recent open-weight diffusion large language models (dLLMs), including LLaDA [27] and its preference-optimized successor LLaDA 1.5 [65], and Dream [55], have scaled the masked-diffusion paradigm to parameter counts and benchmark performance competitive with similarly sized autoregressive LLMs; code-specialized variants such as DiffuCoder [7] and Dream-Coder [52] adapt this paradigm specifically to large code corpora, and both report analyses of how a dLLM’s denoising order and effective causality differ from autoregressive generation. At commercial scale, Mercury [14] and Gemini Diffusion [8] demonstrate that diffusion-based code generation can match leading autoregressive code models on standard coding benchmarks while substantially reducing generation latency, since many token positions can be refined in parallel per denoising step rather than one token at a time. Li et al. [23] survey this broader emerging area, cataloguing architectural variants, training objectives, and open efficiency challenges across text and code applications of diffusion language modeling. We view this literature as a plausible source of future C2Rust methodology – particularly for SACTOR’s idiomatic-refinement phase, which is itself a whole-program revision step – but the present work retains the standard autoregressive formulation throughout, and none of the training or evaluation described below involves a diffusion objective.

3 Base Model

We use **Qwen/Qwen3.5-27B** [33] as the source model for our training. This checkpoint is part of the Qwen3.5 family released by the Qwen Team (Alibaba) in February 2026, and is distinct from the original Qwen3 dense lineup (0.6B/1.7B/4B/8B/14B/32B) released earlier: Qwen3.5 introduces a new hybrid-attention architecture and, in its base release, unified text/vision-language training. Qwen3.5-27B is released under the Apache 2.0 license.

3.1 Architecture

Qwen3.5-27B is natively a causal language model with an attached vision encoder (i.e., a unified multimodal checkpoint), but since our task (C2Rust translation) is purely textual, we use it in text-only / language-model-only mode throughout fine-tuning and inference, bypassing the vision tower. Table 1 summarizes the language-model architecture as specified in the official model card.

Table 1: Qwen3.5-27B language-model architecture (as released).

Property	Value
Parameters (language model)	27B ($\sim$ 28B including the vision encoder)
Hidden dimension	5,120
Number of layers	64
Layer layout	$16 \times [3 \times (\text{Gated DeltaNet} \rightarrow \text{FFN}) \rightarrow 1 \times (\text{Gated Attention} \rightarrow \text{FFN})]$
Gated DeltaNet heads	48 (value), 16 (query/key)
Gated DeltaNet head dimension	128
Gated Attention heads	24 (query), 4 (key/value)
Gated Attention head dimension	256
RoPE dimension (Gated Attention)	64
FFN intermediate dimension	17,408
Token embedding / LM head size	248,320 (padded)
Multi-token prediction (MTP)	Trained with multi-step MTP head
Native context length	262,144 tokens (YaRN-extensible to 1,010,000)
License	Apache 2.0

Architecturally, Qwen3.5-27B is therefore *not* a conventional GQA-only Transformer: three out of every four layers use **Gated DeltaNet**, a gated linear-attention mechanism with a recurrent (constant per-step cost) formulation, while the fourth layer in each 4-layer block uses standard **Gated Attention** with grouped-query attention (24 query heads sharing 4 key/value heads) and rotary position embeddings. This hybrid design is intended to give near-linear-attention efficiency at long context lengths while retaining full attention’s precision at fixed intervals – relevant to our setting since C2Rust translation inputs (a full C source file, possibly with multiple functions/structs) plus SACTOR’s iterative compiler/test feedback can produce long contexts within a single agent turn.

Fine-tuning is performed directly on the released BF16 weights of Qwen/Qwen3.5-27B, with no architectural modifications: the model’s layer layout, Gated DeltaNet/Gated Attention structure, hidden/FFN dimensions, and vocabulary are used unchanged from the official checkpoint (Table 1). Although Qwen3.5-27B is natively a unified vision-language checkpoint, all of our training data (Stages 1–3) is text-only – no image or video inputs are used at any stage – so the vision encoder is exercised only insofar as it remains part of the loaded checkpoint; it receives no gradient signal from our text-only instruction/pretraining data and plays no role in the C2Rust task itself. The vision-encoder weights were not stripped from the checkpoint prior to training; they were left in place but unused, since no image or video data appears anywhere in Stages 1–3. As a result, the full $\sim$ 28B-parameter checkpoint (language model plus vision encoder) is loaded and occupies memory during fine-tuning, even though only the $\sim$ 27B-parameter language-model component receives gradient updates from our text-only training data.

3.2 Tokenizer and Licensing

Qwen3.5-27B uses a padded vocabulary of 248,320 tokens (tied or separately-sized token embedding and LM head, per Table 1). The tokenizer and vocabulary are used unmodified throughout all three training stages – no new special tokens (e.g., for delimiting C/Rust code blocks or instruction fields) were added; instruction formatting relies entirely on the model’s existing chat template and vocabulary. The model is released under the Apache 2.0 license, which permits fine-tuning and redistribution of derivative weights, and no additional Qwen usage-policy terms apply beyond this license for the checkpoint used.

3.3 Fine-Tuning Hardware

All three training stages (§4.1–§4.3) were run on a single node of 8 NVIDIA B300 (Blackwell Ultra) GPUs. Fine-tuning is performed in BF16, matching the precision of the released Qwen3.5-27B weights (no FP8/NVFP4 quantization or mixed-precision downcasting is applied during training, though NVFP4 remains available on Blackwell Ultra should later inference-time serving benefit from it).

3.3.1 Distributed Training Strategy: DeepSpeed ZeRO Stage 3

Distributed training across the 8-GPU node is managed with **DeepSpeed ZeRO Stage 3** [34], which fully partitions optimizer states, gradients, and model parameters across all 8 data-parallel ranks rather than replicating them on every GPU. Given the ample per-GPU memory of the B300 relative to the ~56 GB BF16 footprint of the full Qwen3.5-27B checkpoint (§3), ZeRO-3 partitioning is used purely as a data-parallel memory-efficiency and scaling strategy; no additional tensor-parallel or pipeline-parallel dimension is layered on top. Table 2 summarizes the key settings from our DeepSpeed configuration (`ds_z3_config.json`).

Table 2: DeepSpeed ZeRO Stage 3 configuration used for all three fine-tuning stages.

Setting	Value
ZeRO stage	3
Precision	<code>bf16.enabled = auto</code> (BF16 training)
<code>zero_allow_untested_optimizer</code>	<code>true</code>
<code>overlap_comm</code>	<code>false</code>
<code>contiguous_gradients</code>	<code>true</code>
<code>sub_group_size</code>	1×10^9
<code>stage3_prefetch_bucket_size</code>	<code>auto</code>
<code>stage3_param_persistence_threshold</code>	<code>auto</code>
<code>stage3_max_live_parameters</code>	1×10^9
<code>stage3_max_reuse_distance</code>	1×10^9
<code>stage3_gather_16bit_weights_on_model_save</code>	<code>true</code>

A few settings are worth calling out explicitly:

- **`overlap_comm`:** `false` disables overlapping ZeRO-3’s parameter-gathering/gradient-reduction communication with compute, trading some throughput for reduced GPU memory fragmentation/pressure – a reasonable choice at 27B+ scale even with B300’s abundant memory, since ZeRO-3’s all-gather/reduce-scatter traffic across a full 16-GPU node can still be substantial. `contiguous_gradients: true` keeps gradient buffers contiguous to reduce memory fragmentation, working in tandem with this setting.

- **Large `sub_group_size`, `stage3_max_live_parameters`, and `stage3_max_reuse_distance`** (each 1×10^9) effectively relax ZeRO-3’s default parameter-partitioning granularity, allowing larger groups of parameters to be gathered live at once – a setting that is only comfortably affordable because of the B300’s large per-GPU HBM3e capacity, and would likely need to be lowered on GPUs with less memory headroom.
- **`stage3_gather_16bit_weights_on_model_save: true`** ensures that when a checkpoint is saved, the ZeRO-3-partitioned shards are gathered back into full 16-bit (BF16) weights on a single rank before writing to disk, rather than requiring an offline shard-merging step – consistent with our released checkpoints being BF16 throughout, with no FP32 master-weight export step in the saving pipeline.
- **`zero_allow_untested_optimizer: true`** permits use of an optimizer with ZeRO-3 that DeepSpeed has not explicitly validated for that stage.

4 Training Data and Curriculum

Table 3 summarizes the three training stages. Full dataset statistics are given in the following subsections.

Table 3: Overview of the three-stage training curriculum.

Stage	Objective	Data Source	Training Type
1	Rust language proficiency	Rust code from various Rust datasets	Continued pretraining
2	Debugging / self-repair	<code>microsoft/Verus_Training_Data</code>	Instruction SFT
3	C2Rust task translation	LeetCode C/Rust paired solutions	Instruction SFT

4.1 Stage 1: Rust-Focused Continued Pretraining

To strengthen the base model’s command of Rust prior to instruction tuning, we continue pretraining Qwen3.5-27B on a corpus of Rust-specific text and code assembled entirely from existing, publicly available Hugging Face datasets, rather than from a single source. Rather than drawing this corpus from one Rust-capable model’s outputs, we combine seven complementary Hugging Face sources spanning synthetic multi-task Rust instruction/code data, fill-in-the-middle (FIM) data, competitive-programming program-synthesis and automatic-program-repair (APR) data, general open-source-derived code instructions, canonical function-level benchmarks, and large-scale raw GitHub source code. Each source is processed independently and repackaged into one or more LLaMA-Factory-format pretraining files (organized, where a source has natural sub-categories, into one output file per sub-category), together with a small `dataset_info.json` manifest per source mapping every source onto a single unified `text` column (all instruction-formatted sources are flattened to plain continuation text via direct field concatenation, rather than kept in chat/instruction-response format, consistent with a continued-pretraining rather than instruction-tuning objective at this stage).

4.1.1 Data Composition

Table 4 details the seven source datasets, their original providers, the number of Rust-relevant examples retained from each, and the number of output files each was split into. In total, Stage 1 pretraining draws on **1,673,289** examples.

Table 4: Stage 1 pretraining corpus composition, by Hugging Face source dataset.

Name	Hugging Face Source	Examples
Strandset-Rust	Fortytwo-Network/Strandset-Rust-v1	191,008
CodeFIM-Rust-Mellum	Etherll/CodeFIM-Rust-Mellum	56,920
rust_instruction_dataset	ysr/rust_instruction_dataset	524
humaneval-rust	diversoailab/humaneval-rust	164
Magocoder-OSS-Instruct-75K-rust	ise-uiuc/Magocoder-OSS-Instruct-75K	4,695 (Rust subset)
xCodeEval-rust	NTU-NLP-sg/xCodeEval	39,510
starcoderdata-rust	bigcode/starcoderdata	380,468 (Rust subset)
Total		1,673,289

Strandset-Rust-v1. [15] A large synthetic Rust dataset generated and cross-validated via a multi-small-language-model “swarm” generation-and-peer-review process, spanning 15 task categories including code generation, completion, bug detection, refactoring, optimization, documentation, and testing, with construction grounded in real crates from crates.io and automated compilation/ownership-correctness checks. We split this source by its native `task_category` field into 15 output files, preserving the dataset’s original task-level structure for potential per-category ablation or reweighting.

CodeFIM-Rust-Mellum. [5] A Rust-specific fill-in-the-middle dataset (prefix/suffix/middle triples drawn from 8,628 distinct Rust source files), originally built to fine-tune JetBrains’ Mellum model for Rust code completion. Each example is labeled with one of four FIM types – `random_line_split`, `identifier_name`, `identifier_body`, and `conditional_block` – and we split the source into one output file per type. Including FIM-style data in the pretraining mix is intended to strengthen the model’s ability to reason about partial/incomplete Rust code and surrounding context, a capability plausibly relevant to iterative repair during evaluation.

rust_instruction_dataset. A small, community-curated Rust instruction dataset. Given its small size relative to the other sources, it contributes negligibly to total token count but adds some diversity of instruction phrasing.

humaneval-rust. A human-translated Rust port of the standard 164-problem HumanEval benchmark (function signature, docstring, and canonical solution per problem).

Magocoder-OSS-Instruct-75K, Rust subset. [51] Drawn from the OSS-Instruct dataset, which synthesizes instruction/response coding examples by prompting an LLM (GPT-3.5-turbo-1106) with real open-source code snippets as seeds, intended to produce more diverse and realistic instructions than purely LLM-imagined ones. We retain only the Rust-language subset of this otherwise multilingual dataset.

xCodeEval, Rust subset. [17] xCodeEval is a large multilingual, execution-verified benchmark built from Codeforces competitive-programming problems, covering seven tasks across up to 17 languages. We draw from two of its generative task configurations, restricted to Rust: `program_synthesis` (30,732 examples: natural-language problem description → Rust solution) and `apr` (8,778 examples: buggy Rust source → fixed Rust source, paired with an execution outcome label). Each configuration is written to its

own output file. The APR portion is notable for again providing debugging/repair-style signal, though at the level of competitive-programming logic bugs rather than Verus proof errors (cf. Stage 2, §4.2).

starcoderdata, Rust subset. [1] By far the largest component of the Stage 1 corpus, drawn from StarCoderData – the decontaminated, filtered subset of The Stack (permissively licensed GitHub source code) used to pretrain the StarCoder family of models across 86 programming languages. We retain only the Rust-language portion. This source dominates the total example count and is expected to dominate total token count as well, providing broad, naturalistic exposure to real-world Rust code style and idiom beyond what the smaller, synthetic, or benchmark-derived sources above can offer.

4.1.2 Concatenation Design

A core principle across all seven sources is that pretraining text is built by *directly concatenating a source’s own raw fields*, with **no added fixed headers, template boilerplate, or markdown code fences** (e.g., no synthetic `## Rust Code Review` section titles, and no wrapping code in ````rust` fences) – the goal is text that reads as a natural, continuous document rather than an artificially templated instruction example, since any natural-language fields present (descriptions, explanations, rationale) already provide connective structure between code spans. Table 5 summarizes the concatenation pattern used per data type, with a representative source for each.

Table 5: Concatenation patterns used to build Stage 1 pretraining text, by data type.

Data Type	Concatenation Pattern	Representative Source
FIM / code completion	prefix middle/completion suffix	+ CodeFIM-Rust- + Mellum
Instruction + code Context + code	instruction + code code_context + code	rust_instruction_dataset Several Strandset-Rust-v1 categories
Code + explanation	code_context + code explanation/summary	+ Strandset-Rust-v1 (e.g., documentation-oriented categories)
Before/after (repair-style)	code_context code_before rationale/comment code_after	+ Strandset-Rust-v1 + (e.g., code-review categories)
Full source	prompt + declaration solution + test	+ humaneval-rust

When a source has a natural categorical field (`task_category` for Strandset-Rust-v1; `fim_type` for CodeFIM-Rust-Mellum; task configuration for xCodeEval), the output is split into one JSONL file per category; sources without such a field, or too small to warrant splitting, are written as a single output file. Each `<dataset>-pt/` directory follows a standard layout: `plan.md`, `dataset.schema.md`, `convert_to_pretrain.py`, and a `data/` folder containing `dataset.info.json` plus one or more `*.json` (JSONL) output files.

4.1.3 Training Configuration

Stage 1 is launched via LLaMA-Factory using the YAML recipe summarized in Table 6. Training is **full-parameter** (`finetuning_type: full`, not LoRA or another parameter-efficient method), using the shared DeepSpeed ZeRO-3 configuration described in §3 (`ds_z3_config.json`), in BF16.

Table 6: Stage 1 (continued pretraining) training configuration, from the LLaMA-Factory recipe.

Setting	Value
LLaMA-Factory stage	pt (continued pretraining)
Fine-tuning type	full (full-parameter)
Distributed backend	DeepSpeed ZeRO Stage 3 (§3)
Precision	BF16
<code>max_samples</code>	10^8 (effectively unlimited – the full corpus is used)
Sequence length (<code>cutoff_len</code>)	16,384 tokens
Per-device train batch size	8
Gradient accumulation steps	2
GPU count	8
Learning rate	1.0×10^{-6}
LR scheduler	Cosine, with 5% warmup
Epochs	1.0
Preprocessing / dataloader workers	128 / 8

4.2 Stage 2: Debugging-Aware SFT on Verus_Training_Data

4.2.1 Dataset Origin and Task Nature

`microsoft/Verus_Training_Data` is the training corpus released alongside VeruSyn, a data-synthesis pipeline for *Verus* [21, 20] – a deductive verification tool that lets programmers state pre-/post-conditions for Rust functions (in ordinary Rust syntax) and then discharges the resulting proof obligations to an SMT solver, using programmer-supplied *proof annotations* (e.g., loop invariants, `assert` statements, `decreases` clauses) as hints where automation alone is insufficient [4].

It is important to characterize precisely what “debugging” means in this dataset, since it directly determines what capability Stage 2 transfers to our C2Rust model: the dataset does *not* center on repairing arbitrary runtime logic bugs. Instead, each debugging instance pairs (i) a Rust program annotated with an incomplete or incorrect Verus proof that the Verus prover rejects, together with the corresponding verifier error report, with (ii) a corrected version of the same program whose proof the verifier accepts. The underlying skill being trained is therefore *consuming a formal verifier’s structured failure output and iteratively revising proof annotations (not program logic) until verification succeeds*, while leaving the program’s specification and executable behavior unchanged.

4.2.2 Data Synthesis Pipeline (VeruSyn)

The dataset was constructed because hand-verified Verus code is extremely scarce: at the time of its release, open-source Verus-verified systems totaled well under 200K lines of code and under 1,000 stand-alone verification tasks, far too little to fine-tune an LLM directly [4]. VeruSyn addresses this scarcity with a two-part synthesis pipeline:

- **Part 1 – Self-synthesis and tutorial-based synthesis.** Starting from the $\sim 10,000$ verified programs and $\sim 15,000$ debugging pairs in the SAFE dataset [2] (used to fine-tune an initial Llama-3.3-70B-Instruct “proof generator”), VeruSyn alternates two program-generation strategies over several iterative rounds: (a) *self-synthesis*, where the proof-generation model free-generates new Rust programs together with Verus specifications and proofs from scratch, and (b) *tutorial-based synthesis*, where expert-written seed programs covering each knowledge point in the official Verus Tutorial are expanded by the model into thousands of variants per seed, explicitly to cover Verus features (e.g., quantifiers, `broadcast` facts, nonlinear arithmetic, bit-vector reasoning) that are common in real-world system verification but rare in purely self-generated data. After each round, outputs are deduplicated (via SimHash), checked against the Verus prover, and either kept as *direct-generation* data (if verified on the first attempt) or fed back to the model together with the verifier’s error report to attempt a repair, with successful repairs kept as *debugging* data. This process ultimately yields 6.9 million verified programs (approximately 5.7M direct-generation and 1.2M debugging instances after deduplication and a final compatibility pass against a newer Verus release).
- **Part 2 – Agent trajectory / long chain-of-thought data.** To capture longer, more complex reasoning of the kind needed for real-world (rather than small algorithmic) verification tasks, VeruSyn additionally records full trajectories of a coding agent (GitHub Copilot CLI backed by Claude Sonnet 4.5) attempting to add Verus proofs to real system code, retaining the agent’s interleaved reasoning steps, verifier invocations, and edits. Successful trajectories are split into (i) long, direct-generation chains-of-thought spanning an entire multi-step repair process, and (ii) individual single-step debugging examples extracted from within those trajectories. This yields 4,557 additional CoT instances.

4.2.3 Quality Control

Because a fine-tuned proof-generation model can learn to satisfy the verifier without actually proving the intended property, VeruSyn applies automated anti-cheating filters before accepting any synthesized instance, rejecting programs that: (i) alter the original specification (e.g., quietly moving a postcondition into a precondition), (ii) introduce unjustified escape hatches such as `assume`, `admit`, or `external_body` to bypass proof obligations, or (iii) rely on non-terminating loops to trivially avoid verification of subsequent code [4]. This filtering is reported to be essential – without it, models trained on the resulting data increasingly learn to cheat rather than to prove.

4.2.4 Relevance to This Work

We adopt `microsoft/Verus_Training_Data` as our Stage 2 curriculum because, although its object-level task (formal proof repair) differs from ours (C-to-Rust semantic translation), the *meta-skill* it trains is directly relevant: interpreting a tool’s structured failure output (a Verus error report, analogous to a Rust compiler error or a failing end-to-end test in our evaluation setting) and producing a targeted, minimal revision rather than a wholesale rewrite. Since the SACTOR-style agentic evaluation we use (§5) explicitly drives translation refinement through repeated compiler/test feedback, we hypothesize that this stage improves our model’s ability to make productive use of that feedback loop, even though none of the Stage 2 data itself involves C code or C-to-Rust translation.

Since Stage 2 data is drawn entirely from the Verus/formal-verification domain, it may also teach stylistic or lexical habits (e.g., inserting specification-style comments, favoring proof-annotation-like constructs) that are not appropriate for idiomatic, spec-free application code, and this transfer should be checked empirically rather than assumed.

4.2.5 Dataset Composition

Table 7 summarizes the composition of the Stage 2 corpus as released in `microsoft/Verus_Training_Data`, prior to any subsetting or reformatting we may have applied for our own SFT recipe.

Table 7: Composition of `microsoft/Verus_Training_Data` (VeruSyn dataset), as reported in [4].

Part	Subtype	Approx. Count
Part 1 (self-/tutorial-synthesis)	Direct-generation (program + spec + proof)	5.7M
Part 1 (self-/tutorial-synthesis)	Debugging (unverified $\rightarrow$ verified pair + error report)	1.2M
Part 2 (agent trajectories, Sonnet 4.5)	Long direct-generation CoT	871
Part 2 (agent trajectories, Sonnet 4.5)	Single-step debugging CoT	3,686
Total		$\sim 6.9\text{M} + 4,557$

4.2.6 Training Configuration

Stage 2 is launched via LLaMA-Factory using the YAML recipe summarized in Table 8, following the same overall recipe style as Stage 1 (§4.1): full-parameter fine-tuning under the shared DeepSpeed ZeRO-3 configuration, in BF16.

Table 8: Stage 2 (Verus debugging SFT) training configuration, from the LLaMA-Factory recipe.

Setting	Value
LLaMA-Factory stage	<code>sft</code>
Fine-tuning type	<code>full</code> (full-parameter)
Initialized from	Stage 1 checkpoint
Distributed backend	DeepSpeed ZeRO Stage 3 (§3)
Precision	BF16
<code>max_samples</code>	10^8 (effectively unlimited – the full configured dataset is used)
Sequence length (<code>cutoff_len</code>)	16,384 tokens
Per-device train batch size	12
Gradient accumulation steps	2
GPU count	8
Learning rate	2.0×10^{-7}
LR scheduler	Cosine, with 5% warmup
Epochs	2.0
Preprocessing / dataloader workers	16 / 2

The numerical training settings show Stage 2 using a larger effective batch size (384 vs. 128 for Stage 1) and a lower learning rate over more epochs, consistent with a shorter, more targeted SFT phase following the longer Stage 1 pretraining run.

4.3 Stage 3: C2Rust Task-Specific SFT on the C2Rust-Moxin Dataset

The final stage directly targets the C2Rust translation task using **C2Rust-Moxin** [3], a purpose-built, large-scale paired C/Rust dataset (GitHub: `Bobchenyx/Moxin-C2Rust-Datasets`) constructed specifically for learning, evaluating, and benchmarking C-to-Rust translation. Unlike a single-granularity LeetCode-

only corpus, C2Rust-Moxin is organized into **three granularities** of aligned C $\leftrightarrow$ Rust pairs, each targeting a different level of translation difficulty:

- **functions/** – function-level C $\leftrightarrow$ Rust pairs, aligned one-to-one, supporting fine-grained modeling of syntax conversion and semantic equivalence at the single-function level. This is the granularity most directly comparable to our earlier LeetCode-based description of Stage 3: short, self-contained, algorithmically well-specified translation pairs.
- **programs/** – program-level (CLI or small application) correspondences, enabling evaluation of end-to-end translation and compilation consistency across a complete, runnable program rather than an isolated function.
- **projects/** – project-level C $\leftrightarrow$ Rust mappings across complete, larger codebases, supporting study of large-scale cross-language migration and structural analysis (e.g., multi-file organization, cross-function dependencies) that a single function or small program cannot exercise.

The dataset’s construction draws on and acknowledges several upstream sources: the `c2rust` transliteration tool [13] (likely as a source of rule-based reference translations and/or C source material), the SACTOR project itself [64], and both the international (`leetcode.com`) and Chinese (`leetcode.cn`) LeetCode problem sets.

Within the function-level component specifically, training pairs are constructed from problems (from LeetCode and/or LeetCode-CN) for which both a C solution and a corresponding Rust solution exist; each example pairs the C solution as input with the Rust solution as the target output, optionally accompanied by the problem statement as additional context.

4.3.1 Data Quality Considerations

Since Stage 3 SFT combines `functions/` and `programs/`-level pairs but excludes `projects/`-level (full-codebase) data, the model receives exposure to both short, self-contained algorithmic translations (LeetCode, at the `functions/` granularity) and complete, compilable CLI/small-application translations (at the `programs/` granularity), but not to the multi-file, cross-function structural complexity that only full-codebase examples would provide. This partially mitigates – but does not eliminate – the distribution-mismatch concern raised in earlier drafts of this report: `programs/`-level examples are closer in kind to the SACTOR evaluation targets (§5) than pure function-level pairs would be, since they must compile and run end-to-end rather than being isolated functions, but Stage 3 still provides no training-time exposure to project-scale translation of the kind CRust-Bench and libogg exercise.

4.3.2 Training Configuration

Stage 3 is launched via LLaMA-Factory using the YAML recipe summarized in Table 9, again following the same full-parameter, DeepSpeed ZeRO-3, BF16 recipe style as Stages 1–2.

5 Evaluation Framework

We evaluate the fine-tuned model using an execution-based C-to-Rust evaluation framework built on SACTOR [64]. Rather than relying on textual similarity between generated code and a reference implementation, our evaluation focuses on functional correctness through compilation and execution. A translation is considered successful only when the generated Rust program can be compiled and reproduces the observable behavior of the original C program on the provided test cases.

Table 9: Stage 3 (C2Rust task-specific SFT) training configuration, from the LLaMA-Factory recipe.

Setting	Value
LLaMA-Factory stage	sft
Fine-tuning type	full (full-parameter)
Initialized from	Stage 2 checkpoint
Distributed backend	DeepSpeed ZeRO Stage 3 (§3)
Precision	BF16
max_samples	10^8 (effectively unlimited – the full configured dataset is used)
Sequence length (cutoff_len)	16,384 tokens
Per-device train batch size	10
Gradient accumulation steps	2
GPU count	8
Learning rate	2.0×10^{-7}
LR scheduler	Cosine, with 5% warmup
Epochs	2.0
Preprocessing / dataloader workers	16 / 2

5.1 SACTOR Pipeline

SACTOR organizes C-to-Rust translation as a structured, agentic process. Its complete pipeline can be summarized as follows:

1. **Task division.** The input C program is analyzed and decomposed into translation units, such as data types, global variables, and functions. Dependencies among these units are used to determine an appropriate translation order.
2. **Unidiomatic translation.** Each translation unit is converted into interface-preserving Rust. The resulting code may retain raw pointers, C-style data representations, and `unsafe` operations in order to preserve the behavior of the original C program.
3. **Idiomatic translation.** The unidiomatic Rust implementation is further refined toward safer and more idiomatic Rust, including ownership reconstruction and the replacement of raw pointers where possible.
4. **Verification and repair.** Generated implementations are validated through executable tests. When verification fails, the translation may be revised within a fixed repair budget.

5.2 Evaluation Benchmark

The evaluation benchmark contains 200 C programs, including 92 programs that receive input through command-line arguments (`argv`) and 108 programs that read from standard input. Approximately 120 of these programs are derived from IBM Project CodeNet [31], while the remaining programs were constructed for this benchmark.

Each program is associated with a set of test inputs and reference outputs obtained by compiling and executing the original C program. The evaluation therefore does not depend on a fixed Rust reference implementation or require the generated code to match a particular target solution in textual form or program structure. Instead, the generated Rust program is evaluated according to whether it reproduces the observable behavior of the reference C program on the same test inputs.

The input convention for each program, either `argv` or standard input, is explicitly provided to the translation system. This reduces failures caused by incorrectly inferred input interfaces and prevents interface mismatches from being counted as translation errors.

5.3 Verification and Repair

For each C program, SACTOR constructs the corresponding translation context and invokes the evaluated model to generate Rust code. The generated program is compiled and, if compilation succeeds, executed on the predefined test inputs. Its outputs are then compared with those produced by the reference C program under the same inputs.

A program is considered successfully translated only if the generated Rust code compiles and passes all associated end-to-end tests. If verification fails, the model may perform additional translation attempts within a fixed budget. In the current configuration, each program is allowed up to six translation attempts. A program is counted as a failure if no test-passing Rust implementation is obtained within this budget.

Accordingly, the evaluation measures behavioral agreement on the provided test suite rather than formal semantic equivalence between the generated Rust program and the original C program over all possible inputs.

5.4 Inference Configuration

During evaluation, SACTOR constructs the translation prompt from the input C program and its static-analysis context. Generation uses stochastic sampling: temperature controls the randomness of the output distribution, top-p restricts sampling to the smallest set of tokens whose cumulative probability reaches the specified threshold, and top-k further limits the candidate set to the highest-probability tokens.

The main inference settings are summarized in Table 10.

Table 10: Inference configuration used in evaluation.

Configuration	Value
Temperature	0.6
Top-p	0.95
Top-k	20
Maximum output length	1,536
Maximum translation attempts	6

The maximum output length bounds the amount of code generated in a single model call, while the maximum number of translation attempts defines the repair budget for each program. These settings are held fixed across repeated evaluation runs.

5.5 Failure Analysis

In addition to the final success or failure outcome, the evaluation runner records failure information from execution status and runtime logs. Recorded categories include exhausted translation attempts, execution timeouts, translation-process errors, and certain failures arising during static analysis.

These records do not contribute to the primary evaluation metric, but provide additional information for identifying the sources of unsuccessful translations.

5.6 Evaluation Metric

We use program-level Success Rate (SR) as the primary evaluation metric. For an evaluation run with random seed s , the success rate is defined as

$$SR_s = \frac{N_{\text{pass},s}}{N_{\text{total}}} \times 100\%. \quad (1)$$

Here, $N_{\text{pass},s}$ denotes the number of programs that are successfully translated and pass all end-to-end tests under seed s , while N_{total} denotes the total number of programs in the evaluation benchmark.

Because generation uses stochastic sampling, each model is evaluated using five different random seeds. The final reported success rate is the arithmetic mean of the five seed-level success rates:

$$\overline{SR} = \frac{1}{5} \sum_{s=1}^5 SR_s. \quad (2)$$

This averaged success rate serves as the primary measure of C-to-Rust translation performance in the subsequent experimental results.

6 Results

6.1 Overall SR and Baseline Comparison

Table 11 reports overall accuracy on the C2Rust translation evaluation (§5) for our fine-tuned model alongside six baseline systems, spanning both open-weight models an order of magnitude or more larger than ours and a leading proprietary coding agent.

Table 11: Overall SR on the C2Rust translation evaluation, by model.

Model	Model Size	SR
Qwen3.5-Plus	397B (17B active)	77.20%
MiniMax-M2.5	230B (10B active)	83.90%
GLM-5	744B (40B active)	84.40%
GLM-5.2	744B (40B active)	89.90%
Claude Code-4.6	—	90.01%
Qwen3.5-27B (base, no fine-tuning)	27B (dense)	72.30%
Ours (Qwen3.5-27B + Stages 1–3)	27B (dense)	87.30%

Baselines. The six comparison points span a range of scales and access models:

- **Qwen3.5-Plus** [32] – the production API-hosted variant of Qwen3.5-397B-A17B, Alibaba’s flagship Qwen3.5-family MoE model (397B total / 17B active parameters), sharing the same hybrid Gated-DeltaNet/Gated-Attention architecture family as our own 27B base model (§3) but at roughly 14× the total parameter count.
- **MiniMax-M2.5** [26] – an open-weight (MIT-licensed) MoE model from MiniMax with 230B total / 10B active parameters, positioned as a strong, cost-efficient agentic coding model.

- **GLM-5** [57] and **GLM-5.2** [58] – successive open-weight (MIT-licensed) flagship MoE models from Z.ai (formerly Zhipu AI), both with roughly 744B total / 40B active parameters; GLM-5.2 (released June 2026) is GLM-5’s direct successor and, at the time of writing, among the strongest openly available coding-oriented models.
- **Claude Code-4.6** – Anthropic’s Claude Sonnet 4.6 model accessed through the Claude Code agentic coding harness; as a closed model, its parameter count is not publicly disclosed (– in Table 11).
- **Qwen3.5-27B (base)** – our own source model (§3) evaluated with no fine-tuning at all, serving as the direct before/after reference point for our three-stage curriculum.

Discussion. Two comparisons are most salient. First, relative to its own unmodified starting point, our three-stage curriculum improves accuracy from 72.30% to 87.20% – a 14.9 percentage-point gain – at identical model size and inference cost, indicating that the training curriculum (§4.1–§4.3) rather than raw scale is doing substantial work here. Second, our 27B model outperforms three open-weight models with $8.5\times$ to $27.6\times$ more total parameters (Qwen3.5-Plus at 397B, MiniMax-M2.5 at 230B, and GLM-5 at 744B), suggesting that task-specific specialization can compensate for a large scale disadvantage on this particular benchmark. It falls short, however, of GLM-5.2 (89.90%) and Claude Code-4.6 (90.01%) by roughly 2.7–2.8 percentage points – both of which are either a substantially larger, more recently released open-weight model (GLM-5.2) or a frontier closed model deployed through a mature agentic coding harness (Claude Code-4.6) rather than a from-scratch SACTOR-style evaluation loop.

6.2 General Coding Capability: SWE-bench Verified

The C2Rust accuracy results in Table 11 show how our fine-tuned model performs on the specific task it was trained for. A separate and equally important question is whether the three-stage curriculum’s heavy specialization toward Rust and C2Rust translation comes at the cost of the model’s *general* software-engineering ability. We assess this using **SWE-bench Verified** [16], the human-validated 500-task subset of SWE-bench in which a model must resolve a real-world GitHub issue given the repository and issue description, evaluated on the standard pass@1 patch-resolution protocol – a benchmark with no particular emphasis on Rust or C2Rust and, unlike Table 11, not part of our own curriculum’s target task. Table 12 reports results alongside four baselines.

Table 12: SWE-bench Verified pass@1, by model.

Model	SWE-bench Verified
GPT-5-mini (2025-08-07)	72.0
GPT-OSS-120B	62.0
Qwen3.5-122B-A10B	72.0
Qwen3.5-27B (base, no fine-tuning)	72.4
Ours (Qwen3.5-27B + Stages 1–3)	70.6

Baselines. **GPT-5-mini** (2025-08-07) is OpenAI’s smaller proprietary GPT-5-family model; parameter count is not disclosed. **GPT-OSS-120B** [29] is OpenAI’s open-weight MoE model (117B total / 5.1B active parameters), designed to fit on a single 80GB GPU. **Qwen3.5-122B-A10B** is a mid-tier sibling of our own

base model within the Qwen3.5 family (122B total / 10B active, MoE, same hybrid Gated-DeltaNet/Gated-Attention design as Qwen3.5-27B, §3), roughly $4.5\times$ our total parameter count. **Qwen3.5-27B (base)** is again our own unmodified starting checkpoint, as in Table 11.

Discussion. Our fine-tuned model retains strong general coding ability despite three full-parameter SFT/pre-training stages focused narrowly on Rust and C2Rust: at 70.5%, it comfortably exceeds GPT-OSS-120B (62.0%, at over $4\times$ our parameter count) and sits within roughly 1.5–2 points of GPT-5-mini and the substantially larger Qwen3.5-122B-A10B (72.0% each). Less favorably, our model *underperforms its own unmodified base checkpoint* (70.5% vs. 72.4%, a 1.9-point drop) on this general benchmark. Since Qwen3.5-27B (base) and our model are otherwise identical in architecture and size, this gap is attributable entirely to the three-stage training curriculum, and is consistent with a mild form of *catastrophic forgetting* (or at least capability narrowing) induced by full-parameter fine-tuning concentrated on a narrow domain (Rust pretraining, Verus proof debugging, and C2Rust translation) rather than broad general-purpose software engineering.

7 Conclusion

We presented a three-stage fine-tuning curriculum – Rust-focused continued pretraining, debugging-aware SFT, and C2Rust task-specific SFT – applied to Qwen3-27B for the C-to-Rust translation task, and evaluated the resulting model using the agentic, verification-driven framework of SACTOR [64]. Future work includes extending the task-specific training data beyond LeetCode-style problems to better cover pointer-heavy, real-world C codebases, and conducting a systematic ablation of each training stage’s marginal contribution.

References

- [1] BigCode Project. StarCoderData, 2023. Hugging Face dataset, curated from The Stack v1.2 for training StarCoder/StarCoderBase.
- [2] Tianyu Chen, Shan Lu, Shuai Lu, Yeyun Gong, Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Hao Yu, Nan Duan, Peng Cheng, et al. Automated proof generation for Rust code via self-evolution. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [3] Yixiao Chen. Moxin-C2Rust-Datasets: Moxin’s collection of C2Rust datasets, survey & tech report, 2026. GitHub repository.
- [4] Nongyu Di, Tianyu Chen, Shan Lu, Shuai Lu, Yeyun Gong, Peng Cheng, Jacob R. Lorch, Yuan Yao, and Xiaoxing Ma. Reducing the costs of proof synthesis on Rust systems by scaling up a seed training set, 2026.
- [5] Etherll. CodeFIM-Rust-Mellum, 2025. Hugging Face dataset.
- [6] Muhammad Farrukh, Smeet Shah, Baris Coskun, Tapti Palit, and Michalis Polychronakis. SafeTrans: LLM-assisted transpilation from C to Rust, 2025.
- [7] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. DiffuCoder: Understanding and improving masked diffusion models for code generation, 2025. arXiv:2506.20639.
- [8] Google DeepMind. Gemini diffusion, 2025.

- [9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [10] Jaemin Hong and Sukyoung Ryu. Automatically translating C to Rust. *Communications of the ACM*, 68(11):58–65, 2025.
- [11] Jaemin Hong and Sukyoung Ryu. Type-migrating C-to-Rust translation using a large language model. *Empirical Software Engineering*, 30(1):3, 2025.
- [12] Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. AlphaTrans: A neuro-symbolic compositional approach for repository-level code translation and validation. *Proceedings of the ACM on Software Engineering (FSE)*, 2:2454–2476, 2025.
- [13] Immuant. C2rust, 2020. Software.
- [14] Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, and Stefano Ermon. Mercury: Ultra-fast language models based on diffusion, 2025. *arXiv:2506.17298*.
- [15] Aleksei Ivashov, Vladyslav Larin, Vishesh Tripathi, and Ivan Nikitin. Strand-rust-coder-v1: Rust coding model fine-tuned on peer-ranked synthetic data, October 2025.
- [16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [17] Mohammad Abdullah Matin Khan, M Saiful Bari, Xuan Long Do, Weishi Wang, Md Rizwan Parvez, and Shafiq Joty. xCodeEval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2023.
- [18] Anirudh Khatri, Robert Zhang, Jia Pan, Ziteng Wang, Qiaochu Chen, Greg Durrett, and Isil Dillig. CRust-Bench: A comprehensive benchmark for C-to-safe-Rust transpilation, 2025.
- [19] Zhenglun Kong, Yize Li, Fanhu Zeng, et al. Token reduction should go beyond efficiency in generative models – from vision, language to multimodality. *arXiv preprint arXiv:2505.18227*, 2025.
- [20] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, et al. Verus: A practical foundation for systems verification. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, pages 438–454, 2024.
- [21] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. In *Proceedings of the ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 286–315, 2023.
- [22] Ruishi Li, Bo Wang, Tianyu Li, Prateek Saxena, and Ashish Kundu. Translating C to Rust: Lessons from a user study, 2024.

- [23] Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. A survey on diffusion language models, 2025. *arXiv:2508.10875*.
- [24] Xiaofan Liu, Zecan Li, Zhuang Zhao, Ziqi Shuai, Yanming Yang, Qi Xin, and Jifeng Xuan. AdaTrans: Automated C to Rust transformation via error-adaptive repair, 2026.
- [25] Zhendong Mi, Yixiao Chen, Pu Zhao, et al. Effective moe-based llm compression by exploiting heterogeneous inter-group experts routing frequency and information density. *arXiv preprint arXiv:2602.09316*, 2026.
- [26] MiniMax. MiniMax-M2.5, February 2026.
- [27] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models (LLaDA), 2025.
- [28] Vikram Nitin, Rahul Krishna, Luiz Lemos do Valle, and Baishakhi Ray. C2SaferRust: Transforming C projects into safer Rust with neurosymbolic techniques, 2025.
- [29] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025.
- [30] Guangsheng Ou, Mingwei Liu, Yuxuan Chen, Xin Peng, and Zibin Zheng. Repository-level code translation benchmark targeting Rust, 2024.
- [31] Ruchir Puri, David Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks, 2021.
- [32] Qwen Team. Qwen3.5-Plus, February 2026. Alibaba Cloud Model Studio API, backed by Qwen3.5-397B-A17B.
- [33] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.
- [34] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.
- [35] Xuan Shen, Chenxia Han, Yufa Zhou, et al. Draftattention: Fast video diffusion via low-resolution attention guidance. *arXiv preprint arXiv:2505.14708*, 2025.
- [36] Xuan Shen, Weize Ma, Jing Liu, et al. Quartdepth: Post-training quantization for real-time depth estimation on the edge. In *CVPR*, 2025.
- [37] Xuan Shen, Weize Ma, Yufa Zhou, et al. Fastcar: Cache attentive replay for fast auto-regressive video generation on the edge. In *ICLR*, 2026.
- [38] Xuan Shen, Zhao Song, Yufa Zhou, et al. Lazydit: Lazy learning for the acceleration of diffusion transformers. *AAAI*, 39(19), Apr. 2025.
- [39] Xuan Shen, Zhao Song, Yufa Zhou, et al. Numerical pruning for efficient autoregressive models. *AAAI*, 39(19), Apr. 2025.

- [40] Xuan Shen, Yizhou Wang, et al. Efficient reasoning with hidden thinking. *arXiv preprint arXiv:2501.19201*, 2025.
- [41] Xuan Shen, Pu Zhao, Yifan Gong, et al. Search for efficient large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [42] Xuan Shen, Hangyu Zheng, Yifan Gong, et al. Sparse learning for state space models on mobile. In *ICLR*, 2025.
- [43] Manish Shetty, Naman Jain, Adwait Godbole, Sanjit A. Seshia, and Koushik Sen. Syzygy: Dual code-test C to (safe) Rust translation using LLMs and dynamic analysis, 2024.
- [44] HoHyun Sim, Hyeonjoong Cho, Yeonghyeon Go, Zhoulai Fu, Ali Shokri, and Binoy Ravindran. Search-based multi-trajectory refinement for safe C-to-Rust translation with large language models, 2025. Also circulated as “Large Language Model-Powered Agent for C to Rust Code Translation”.
- [45] Mukul Singh, José Cambronero, Sumit Gulwani, Vu Le, Carina Negreanu, and Gust Verbruggen. CodeFusion: A pre-trained diffusion model for code generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [46] Marc Szafraniec, Baptiste Roziere, Hugh Leather, Francois Charton, Patrick Labatut, and Gabriel Synnaeve. Code translation with compiler representations. In *International Conference on Learning Representations (ICLR)*, 2023.
- [47] Amir Taherin, Juyi Lin, Arash Akbari, Arman Akbari, Pu Zhao, Weiwei Chen, David Kaeli, and Yanzhi Wang. Cross-platform scaling of vision-language-action models from edge to cloud gpus. *arXiv preprint arXiv:2509.11480*, 2025.
- [48] Qwen Team. Qwen3. 5-omni technical report. *arXiv preprint arXiv:2604.15804*, 2026.
- [49] Chaofan Wang, Tingrui Yu, Beijun Shen, Jie Wang, Dong Chen, Wenrui Zhang, Yuling Shi, Chen Xie, and Xiaodong Gu. EvoC2Rust: A skeleton-guided framework for project-level C-to-Rust translation. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026. *arXiv:2508.04295*.
- [50] Shengbo Wang, Mingwei Liu, Guangsheng Ou, Yuwen Chen, Zike Li, Yanlin Wang, and Zibin Zheng. His2Trans: A skeleton-first framework for self-evolving C to Rust translation with historical retrieval, 2026.
- [51] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with OSS-Instruct, 2024. *ICML 2024*.
- [52] Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, et al. Dream-coder 7b: An open diffusion language model for code, 2025. *arXiv:2509.01142*.
- [53] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. CodeTransOcean: A comprehensive multilingual benchmark for code translation, 2023.
- [54] Aidan Z. H. Yang, Yoshiki Takashima, Brandon Paulsen, Josiah Dodds, and Daniel Kroening. VERT: Polyglot verified equivalent Rust transpilation with large language models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1453–1463, 2025.

- [55] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025. arXiv:2508.15487.
- [56] Zhiqiang Yuan, Wenjun Mao, Zhuo Chen, Xiyue Shang, Chong Wang, Yiling Lou, and Xin Peng. Project-level C-to-Rust translation via synergistic integration of knowledge graphs and large language models, 2025.
- [57] Z.ai (Zhipu AI). GLM-5, February 2026.
- [58] Z.ai (Zhipu AI). GLM-5.2: A 1m-context open-weight coding model, June 2026.
- [59] Zheng Zhan, Yushu Wu, Yifan Gong, et al. Fast and memory-efficient video diffusion using streamlined inference. In *Fast and Memory-Efficient Video Diffusion Using Streamlined Inference*, volume 37, 2024.
- [60] Zheng Zhan, Yushu Wu, Zhenglun Kong, et al. Rethinking token reduction for state space models. In *EMNLP*. ACL, nov 2024.
- [61] Pu Zhao, Arash Akbari, Xuan Shen, et al. Open-source multimodal moxin models with moxin-vlm and moxin-vla. *arXiv preprint arXiv:2512.22208*, 2025.
- [62] Pu Zhao, Xuan Shen, Zhenglun Kong, Yixin Shen, Sung-En Chang, Arash Akbari, Timothy Rupprecht, Lei Lu, Enfu Nan, Changdi Yang, et al. 7b fully open source moxin-llm/vlm—from pretraining to grpo-based reinforcement learning enhancement. *arXiv preprint arXiv:2412.06845*, 2024.
- [63] Pu Zhao, Fei Sun, Xuan Shen, et al. Pruning foundation models for high accuracy without retraining. In *Findings of EMNLP 2024*, pages 9681–9694. ACL, November 2024.
- [64] Tianyang Zhou, Ziyi Zhang, Haowen Lin, Somesh Jha, Mihai Christodorescu, Kirill Levchenko, and Varun Chandrasekaran. SACTOR: LLM-driven correct and idiomatic C to Rust translation with static analysis and FFI-based verification, 2025.
- [65] Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. LLaDA 1.5: Variance-reduced preference optimization for large language diffusion models, 2025.